

Praktická zkouška

Protokol sanitace podzemních odvodních tras

Popis úlohy

Dokončete REST API systém pro evidenci čištění kanalizačních tras.

Architektura

```
Klient -> Router (port 8000) -> Server 1 (Python, port 8001)
                                -> Server 2 (JS, port 8002)
```

Datový model (db.json)

Entita: **protocol**

```
{
  "id": number,
  "section_code": string, // kod useku (napr. KAN-A7-012)
  "street": string, // ulice
  "district": string, // mestska cast
  "length_m": number, // delka useku v metrech
  "sanitation_type": string, // typ sanitace (pressure, mechanical, camera, chemical)
  "status": string, // stav (completed, pending, blocked, needs_repair)
  "technician": string|null, // jmeno technika
  "date": string|null, // datum sanitace (YYYY-MM-DD)
  "notes": string|null // poznamky
}
```

Požadované REST API endpointy

```
/<entita><DOPLNTE> - vrati vsechny zaznamy
/<entita><DOPLNTE>?status=X - vrati zaznamy filtrovane podle stavu
/<entita><DOPLNTE> - vrati zaznam podle ID
/<entita><DOPLNTE> - vytvori novy zaznam
/<entita><DOPLNTE> - smaze zaznam podle ID
```

Příklad odpovědi

```
GET /protocols/1

HTTP/1.1 200 OK
X-Server-ID: 1
Content-Type: application/json

{
  "id": 1,
  "section_code": "KAN-A7-012",
  "street": "Vodickova",
  "district": "Praha 1",
  "length_m": 145,
  "sanitation_type": "pressure",
  "status": "completed",
  "technician": "Jan Novak",
  "date": "2026-01-15",
  "notes": "Nalezen mensi korenovy narust"
}
```

```
DELETE /protocols/1  
  
HTTP/1.1 204 No Content
```

Každá odpověď musí obsahovat HTTP hlavičku **X-Server-ID: N** (kde N je číslo serveru 1 nebo 2).

Při vytváření záznamu server ignoruje **id** v těle požadavku a generuje vlastní.

DELETE vrací status **204 No Content** při úspěchu nebo **404 Not Found**.

Filtrování **?status=X** musí fungovat na všech serverech i přes router.

Neměňte porty serverů!

Úkoly k vypracování

1. Server 1 (server1.py) – Python Flask [30 bodů]

Implementujte všechny požadované REST API endpointy včetně GET s filtrováním podle **?status=X** a DELETE endpoint.

2. Server 2 (server2.js) – Express.js [30 bodů]

Zkontrolujte a opravte případné chyby. Doplňte chybějící funkcionalitu včetně GET s filtrováním podle **?status=X** a DELETE endpoint.

3. Router (router.py) – Load Balancer [40 bodů]

Implementujte **Ratio 2:3** load balancing (2 požadavky na Python, 3 na JavaScript).

Sekvence: PY, PY, JS, JS, JS, PY, PY, JS, JS, JS, ...

Router musí podporovat všechny HTTP metody včetně DELETE a předávat query parametry.

Testování

```
# Vsechny protokoly  
curl -i http://localhost:8000/<entita><DOPLNTE>  
  
# Filtrovani podle stavu  
curl -i "http://localhost:8000/<entita><DOPLNTE>?status=completed"  
  
# Smazani protokolu  
curl -i -X DELETE http://localhost:8000/<entita><DOPLNTE>/1
```

Hodnocení

Celkem: **100 bodů**. Za každou chybu: -11 bodů.

Odevzdání

server1.py, server2.js, router.py